

SGEMM Performance: Week 3 Optimizations

Core Micro-Kernel Upgrades & Fixing Macro Limitations

TER HPC Project

April 2026

Introduction: Week 3 Objectives

Context: In Week 3, we targeted drastic improvements to our Week 2 SGEMM baseline through two distinct phases:

Phase 1: Core Micro-Kernel Optimizations

- Extending Loop Unrolling ($\times 4$)
- Next-Tile Software Prefetching
- Non-Temporal Stores for the C Matrix

Phase 2: Fixing Structural Limitations

- **Limitation 10.1:** Sequential reduction loop causing bottlenecks.
- **Limitation 10.2:** Heap-allocation overhead dominating small matrices.
- **Limitation 10.3:** Idle threads leading to poor thread scaling at 256^3 .

Phase 1: Deep Micro-Kernel Upgrades

1. Loop Unrolling $\times 4$

- Extended the main k-loop unrolling from $\times 2$ to $\times 4$.
- *Impact:* Reduced loop branch overhead and gave the out-of-order execution engine a much larger instruction window to schedule AVX2 FMAs and memory loads natively.

2. Next-Tile Software Prefetching

- Added software prefetch hints (`_mm_prefetch` with `_MM_HINT_T0`).
- *Impact:* Overlaps the loading of the *next* matrix tile from L2 into L1 cache while the CPU computes the *current* tile, effectively hiding the $\sim 10 - 15$ cycle latency penalty on tile transitions.

3. Non-Temporal Buffer Writes (`_mm256_stream_ps`)

- The resulting C matrix rows are written at the end of a kernel loop and not read again immediately.
- Regular stores pollute the L1/L2 caches with C 's data, evicting the crucial A and B tiles we just prefetched.
- *Impact:* Non-temporal stores bypass the cache hierarchy and stream writes directly to main memory, keeping our A and B panels safely cached for the next iterations. Extremely effective on large matrices (e.g., 2048^3).

Intermediate Results: Phase 1 Impact

The combination of deeper unrolling, prefetching, and non-temporal streaming yielded massive baseline shifts before we even touched macro limitations.

Matrix Dimensions	Threads	Week 2 Baseline	Phase 1 Core	Gains
256 × 256	16	167.61 GF/s	217.96 GF/s	+30.0%
512 × 512	8	176.53 GF/s	373.26 GF/s	+111.4%
1024 × 1024	8	184.13 GF/s	385.63 GF/s	+109.4%
1024 × 1024	16	280.82 GF/s	568.15 GF/s	+102.3%

Core upgrades essentially doubled performance directly across standard boundaries (up to 111% jump).

Phase 2: Fixing Limitation 10.2 (Small Matrix Overhead)

The Problem: For matrices smaller than 128×128 , delegating memory allocation to standard `aligned_alloc` causes heap overhead and OS thread dispatches to dominate thread runtime instead of math.

The Solution: *VLA On-Stack Bypass.*

- Implement an isolation filter threshold for combinations $< 128^3$.
- Allocate arrays natively to the stack via C Standard **Variable Length Arrays (VLAs)**.

Result: Performance on 64^3 matrices jumped by an additional **+24.1%** (34.1 to 42.3 GF/s) by allowing the micro-kernel to handle raw AVX2 vectors instantly on the stack.

Phase 2: Fixing Limitation 10.1 (TASK2 Reduction)

The Problem: TASK2 heavily parallelizes matrix slices across a K-replication system. However, recombining these partial 'C' threads was bottlenecked sequentially (processing $\sim 500\text{K}$ floats behind a single-thread barrier).

The Solution: *OpenMP Nested Parallelization.*

- Converted the summation barrier to utilize `#pragma omp for collapse(2)`.
- Distributes memory bandwidth combinations over the OpenMP thread suite natively.

Result: TASK2 architectures alleviated serial deadlock, boosting output yields for 1024^3 matrices by **+18.0%** sequentially during recombination.

Phase 2: Fixing Limitation 10.3 (Thread Scaling)

The Problem: At 256^3 sizes and $MC=120$, matrix slices only equate to 2 m -strips. On 16-20 thread hardware, the vast majority of threads remain utterly idle while 2 threads handle 100% of the load.

The Solution: *Dynamic Thread Spread Algorithm.*

- If $M / MC < \text{nthd} \times 2$, automatically throttle hardware MC array footprints downward.
- Prevents unbalanced thread-starving by guaranteeing a baseline minimum of micro-loops per available core.

Result: Rebalancing arrays scaled mid-tier throughput by a staggering **+68.0%** at 128^3 and **+18.0%** at 256^3 configurations.

Final Combined Results vs OpenBLAS

Combined, our Core Micro-Kernel Upgrades and targeted Limitation Fixes transformed our SGEMM framework into a highly optimized solution substantially outpacing OpenBLAS targets.

Dimension	OpenBLAS	Week 3 Final (GF/s)	Ratio
64×64	49.62 GF/s	42.10	0.84×
128×128	79.44 GF/s	126.64	1.59×
256×256	151.65 GF/s	328.48	2.16×
512×512	381.20 GF/s	521.36	1.36×
1024×1024	493.58 GF/s	690.74	1.40×
2048×2048	535.75 GF/s	840.69	1.56×